

API REST avec Node.js

Express · PostgreSQL · Zod · Swagger



Node.js 18+

Express 5

PostgreSQL 16

Zod 4

Swagger

TABLE DES MATIÈRES

- 1 Qu'est-ce qu'une API REST ?
- 2 Mise en place du projet
- 3 Connexion à PostgreSQL
- 4 Validation avec Zod
- 5 La ressource Products
- 6 La ressource Users — Exercice 1
- 7 Intégration d'un service externe — Exercice 2
- 8 Recherche et filtrage — Exercice 3
- 9 La ressource Orders — Exercice 4
- 10 La ressource Reviews — Exercice 5
- 11 Documentation Swagger — Exercice 6
- 12 Résumé des bonnes pratiques

Table des matières

1. [Qu'est-ce qu'une API REST ?](#1-quest-ce-quune-api-rest-)
2. [Mise en place du projet](#2-mise-en-place-du-projet)
3. [Connexion à PostgreSQL](#3-connexion-à-postgresql)
4. [Validation avec Zod](#4-validation-avec-zod)
5. [La ressource Products](#5-la-ressource-products)
6. [La ressource Users — Exercice 1](#6-la-ressource-users--exercice-1)
7. [Intégration d'un service externe — Exercice 2](#7-integration-dun-service-externe--exercice-2)
8. [Recherche et filtrage — Exercice 3](#8-recherche-et-filtrage--exercice-3)
9. [La ressource Orders — Exercice 4](#9-la-ressource-orders--exercice-4)
10. [La ressource Reviews — Exercice 5](#10-la-ressource-reviews--exercice-5)
11. [Documentation Swagger — Exercice 6](#11-documentation-swagger--exercice-6)
12. [Résumé des bonnes pratiques](#12-resume-des-bonnes-pratiques)

1. Qu'est-ce qu'une API REST ?

REST (**R**epresentational **S**tate **T**ransfer) est un style d'architecture pour concevoir des services web. Une API REST repose sur le protocole HTTP et organise les données en **ressources**.

Les méthodes HTTP

Chaque action sur une ressource correspond à une méthode HTTP précise :

Méthode	Action	Exemple
GET	Lire une ou plusieurs données	GET /products
POST	Créer une nouvelle donnée	POST /products
PUT	Remplacer entièrement	PUT /products/1
PATCH	Modifier partiellement	PATCH /products/1
DELETE	Supprimer	DELETE /products/1

Les codes de statut HTTP

Le serveur répond toujours avec un code qui indique le résultat :

Code	Signification
200	OK — Succès
400	Bad Request — La requête est invalide
404	Not Found — La ressource n'existe pas

409	Conflict — Conflit (ex : doublon)
502	Bad Gateway — Erreur d'un service en amont

2. Mise en place du projet

Initialisation

```
mkdir REST-POSTGRES
cd REST-POSTGRES
npm init -y
npm install express postgres zod swagger-jsdoc swagger-ui-express
```

BASH

- **express** : framework web pour créer les routes HTTP
- **postgres** : librairie pour communiquer avec PostgreSQL
- **zod** : librairie de validation des données
- **swagger-jsdoc / swagger-ui-express** : génération de documentation automatique

La base de données PostgreSQL avec Docker

On lance PostgreSQL dans un conteneur Docker. L'option **-v** monte le fichier **init.sql** qui sera automatiquement exécuté au démarrage pour créer les tables.

```
docker run --name postgres -p 5432:5432 \
-e POSTGRES_USER=user \
-e POSTGRES_PASSWORD=password \
-e POSTGRES_DB=mydb \
-v ./init.sql:/docker-entrypoint-initdb.d/init.sql \
-d postgres
```

BASH

Pour relancer proprement (supprimer l'ancien conteneur d'abord) :

```
docker container rm postgres -f
```

BASH

Le fichier init.sql

Ce fichier définit toutes les tables de la base de données. Il est exécuté une seule fois au premier démarrage du conteneur.

SQL

```
CREATE TABLE IF NOT EXISTS products (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  about TEXT NOT NULL,  
  price DECIMAL(10, 2) NOT NULL CHECK (price > 0),  
  review_ids INTEGER[] DEFAULT '{}',  
  average_score DECIMAL(3, 2) DEFAULT 0  
);  
  
CREATE TABLE IF NOT EXISTS users (  
  id SERIAL PRIMARY KEY,  
  username VARCHAR(255) UNIQUE NOT NULL,  
  password VARCHAR(128) NOT NULL, -- SHA512 = 128 caractères hexadécimaux  
  email VARCHAR(255) UNIQUE NOT NULL  
);  
  
CREATE TABLE IF NOT EXISTS orders (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  product_ids INTEGER[] NOT NULL DEFAULT '{}',  
  total DECIMAL(10, 2) NOT NULL,  
  payment BOOLEAN NOT NULL DEFAULT FALSE,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE IF NOT EXISTS reviews (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  product_id INTEGER NOT NULL REFERENCES products(id) ON DELETE CASCADE,  
  score INTEGER NOT NULL CHECK (score >= 1 AND score <= 5),  
  content TEXT NOT NULL,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

Points importants :

- **SERIAL PRIMARY KEY** : identifiant entier auto-incrémenté
- **UNIQUE** : interdit les doublons pour ce champ
- **REFERENCES** : clé étrangère — lie une table à une autre
- **ON DELETE CASCADE** : si l'utilisateur est supprimé, ses commandes/avis le sont aussi
- **DEFAULT CURRENT_TIMESTAMP** : la date est remplie automatiquement à l'insertion

3. Connexion à PostgreSQL

Au début de `server.js`, on importe toutes les librairies et on initialise la connexion à la base de données.

```
const express = require("express");
const postgres = require("postgres");
const z = require("zod");
const crypto = require("crypto"); // module natif Node.js, pas besoin de l'installer

const app = express();
const port = 8000;

// Connexion à PostgreSQL
const sql = postgres({ db: "mydb", user: "user", password: "password" });

// Permet à Express de lire le JSON dans le body des requêtes
app.use(express.json());
```

JAVASCRIPT

La librairie **postgres** utilise des **template literals** pour écrire les requêtes SQL de manière sécurisée. Les variables sont automatiquement échappées pour éviter les injections SQL :

```
// Sécurisé : la librairie gère les paramètres
const product = await sql`SELECT * FROM products WHERE id = ${req.params.id}`;

// JAMAIS faire ça (injection SQL possible) :
// const product = await sql`SELECT * FROM products WHERE id = ` + req.params.id`
```

JAVASCRIPT

La requête retourne toujours un **tableau** de résultats. Pour récupérer un seul élément on prend `[0]`.

4. Validation avec Zod

REST n'ayant pas de système de schéma intégré (contrairement à SOAP), on utilise **Zod** pour valider les données envoyées par le client avant de les traiter.

Définir un schéma

```
const ProductSchema = z.object({
  id: z.number(),
  name: z.string(),
  about: z.string(),
  price: z.number().positive(), // doit être un nombre positif
});
```

JAVASCRIPT

Créer des variantes du schéma

Pour la création d'un produit, l'**id** n'existe pas encore — on l'exclut avec `.omit()` :

```
const CreateProductSchema = ProductSchema.omit({ id: true });
```

JAVASCRIPT

Pour une mise à jour partielle (PATCH), on rend tous les champs optionnels avec `.partial()` :

```
const PatchUserSchema = CreateUserSchema.partial().refine(
  (data) => Object.keys(data).length > 0,
  { message: "At least one field must be provided" }
);
// .refine() ajoute une contrainte custom : au moins un champ doit être présent
```

JAVASCRIPT

Utiliser safeParse

safeParse valide les données **sans lever d'exception**. Il retourne un objet avec **success: true/false** et **data** ou **error** :

```
app.post("/products", async (req, res) => {
  const result = CreateProductSchema.safeParse(req.body);

  if (!result.success) {
    // Zod décrit précisément quelle règle a échoué
    return res.status(400).send(result);
  }

  // Ici, result.data est garanti valide
  const { name, about, price } = result.data;
  // ...
});
```

JAVASCRIPT

5. La ressource Products

Structure complète des routes Products

GET	/products	→ liste tous les produits (+ filtres en Exercice 3)
GET	/products/:id	→ récupère un produit par son ID (+ reviews en Exercice 5)
POST	/products	→ crée un nouveau produit
DELETE	/products/:id	→ supprime un produit

JAVASCRIPT

GET /products — lire tous les produits

```
app.get("/products", async (req, res) => {
  const products = await sql`SELECT * FROM products`;
  res.send(products);
});
```

JAVASCRIPT

GET /products/:id — lire un produit

:id est un **paramètre de route** accessible via **req.params.id**.

```
app.get("/products/:id", async (req, res) => {
  const product = await sql`SELECT * FROM products WHERE id = ${req.params.id}`;

  if (!product.length) {
    return res.status(404).send({ message: "Not found" });
  }

  res.send(product[0]);
});
```

JAVASCRIPT

Le `return` avant `res.status(404)` est important : il stoppe l'exécution de la fonction. Sans lui, le code continuerait et enverrait une deuxième réponse, ce qui causerait une erreur.

POST /products — créer un produit

```
app.post("/products", async (req, res) => {
  const result = CreateProductSchema.safeParse(req.body);

  if (!result.success) {
    return res.status(400).send(result);
  }

  const { name, about, price } = result.data;

  const product = await sql`
    INSERT INTO products (name, about, price)
    VALUES (${name}, ${about}, ${price})
    RETURNING *
  `;
  // RETURNING * demande à PostgreSQL de retourner la ligne insérée

  res.send(product[0]);
});
```

JAVASCRIPT

DELETE /products/:id — supprimer un produit

Par convention REST, on retourne la ressource **qui vient d'être supprimée** :

```
app.delete("/products/:id", async (req, res) => {
  const product = await sql`
    DELETE FROM products WHERE id = ${req.params.id} RETURNING *
  `;

  if (!product.length) {
    return res.status(404).send({ message: "Not found" });
  }

  res.send(product[0]);
});
```

JAVASCRIPT

6. La ressource Users — Exercice 1

Règles importantes

1. Ne jamais retourner le mot de passe dans les réponses
2. Ne jamais stocker le mot de passe en clair — on utilise SHA-512

Le hashage SHA-512

SHA-512 est une fonction de **hachage cryptographique** : elle transforme n'importe quelle chaîne en une empreinte de 128 caractères hexadécimaux. Ce processus est **irréversible** — on ne peut pas retrouver le mot de passe original depuis le hash.

Node.js fournit le module **crypto** nativement :

```
const crypto = require("crypto");

const hashPassword = (password) =>
  crypto.createHash("sha512").update(password).digest("hex");

// Exemple :
// hashPassword("motdepasse123")
// → "d404559f602eab6fd602ac7680dacbfaadd13630335e951f097af3900e9de176..."
```

JAVASCRIPT

Les schémas Zod pour Users

```
const UserSchema = z.object({
  id: z.number(),
  username: z.string().min(1),
  password: z.string().min(8), // minimum 8 caractères
  email: z.string().email(), // format email valide
});

const CreateUserSchema = UserSchema.omit({ id: true });

// Pour PATCH : tous les champs sont optionnels mais au moins un requis
const PatchUserSchema = CreateUserSchema.partial().refine(
  (data) => Object.keys(data).length > 0,
  { message: "At least one field must be provided" }
);
```

JAVASCRIPT

POST /users — créer un utilisateur

On hash le mot de passe avant insertion. On gère aussi l'erreur de doublon PostgreSQL (code **23505**) :

JAVASCRIPT

```
app.post("/users", async (req, res) => {
  const result = CreateUserSchema.safeParse(req.body);

  if (!result.success) {
    return res.status(400).send(result);
  }

  const { username, password, email } = result.data;

  try {
    const user = await sql`
      INSERT INTO users (username, password, email)
      VALUES (${username}, ${hashPassword(password)}, ${email})
      RETURNING id, username, email
      -- On NE sélectionne PAS password dans le RETURNING
    `;
    res.send(user[0]);
  } catch (err) {
    if (err.code === "23505") {
      // Code PostgreSQL pour violation de contrainte UNIQUE
      return res.status(409).send({ message: "Username or email already taken" });
    }
    throw err; // Autre erreur inattendue, on la propage
  }
});
```

GET /users — ne jamais exposer le mot de passe

On utilise `SELECT id, username, email` au lieu de `SELECT *` :

JAVASCRIPT

```
app.get("/users", async (req, res) => {
  const users = await sql`SELECT id, username, email FROM users`;
  res.send(users);
});
```

PUT vs PATCH

PUT remplace la ressource entièrement — tous les champs sont obligatoires :

JAVASCRIPT

```
app.put("/users/:id", async (req, res) => {
  const result = CreateUserSchema.safeParse(req.body); // tous les champs requis

  if (!result.success) {
    return res.status(400).send(result);
  }

  const { username, password, email } = result.data;

  try {
    const user = await sql`
      UPDATE users
      SET username = ${username},
          password = ${hashPassword(password)},
          email = ${email}
      WHERE id = ${req.params.id}
      RETURNING id, username, email
    `;

    if (!user.length) {
      return res.status(404).send({ message: "Not found" });
    }

    res.send(user[0]);
  } catch (err) {
    if (err.code === "23505") {
      return res.status(409).send({ message: "Username or email already taken" });
    }
    throw err;
  }
});
```

PATCH met à jour partiellement — seuls les champs envoyés sont modifiés. La librairie **postgres** permet de générer dynamiquement la clause **SET** avec **sql(object, ...keys)** :

JAVASCRIPT

```
app.patch("/users/:id", async (req, res) => {
  const result = PatchUserSchema.safeParse(req.body);

  if (!result.success) {
    return res.status(400).send(result);
  }

  const updates = result.data;

  // Si le mot de passe est modifié, on le hash avant d'envoyer en BDD
  if (updates.password) {
    updates.password = hashPassword(updates.password);
  }

  try {
    const user = await sql`
      UPDATE users
      SET ${sql(updates, ...Object.keys(updates))}
      WHERE id = ${req.params.id}
      RETURNING id, username, email
    `;
    // sql(updates, 'email') génère : email = $1
    // sql(updates, 'username', 'email') génère : username = $1, email = $2

    if (!user.length) {
      return res.status(404).send({ message: "Not found" });
    }

    res.send(user[0]);
  } catch (err) {
    if (err.code === "23505") {
      return res.status(409).send({ message: "Username or email already taken" });
    }
    throw err;
  }
});
```

7. Intégration d'un service externe — Exercice 2

On peut appeler n'importe quel autre service web depuis notre API avec `fetch` (natif dans Node.js depuis la version 18).

L'API **FreeToGame** fournit une liste de jeux gratuits. Notre rôle est de l'exposer via notre propre route.

JAVASCRIPT

```
// GET /f2p-games → retourne tous les jeux F2P
app.get("/f2p-games", async (req, res) => {
  const response = await fetch("https://www.freetogame.com/api/games");

  if (!response.ok) {
    // Le service externe a échoué → on retourne 502 Bad Gateway
    return res.status(502).send({ message: "Failed to fetch F2P games" });
  }

  const games = await response.json();
  res.send(games);
});

// GET /f2p-games/:id → retourne un jeu F2P par son ID
app.get("/f2p-games/:id", async (req, res) => {
  const response = await fetch(
    `https://www.freetogame.com/api/game?id=${req.params.id}`
  );

  if (response.status === 404) {
    return res.status(404).send({ message: "Not found" });
  }

  if (!response.ok) {
    return res.status(502).send({ message: "Failed to fetch game" });
  }

  const game = await response.json();
  res.send(game);
});
```

■ ****Pourquoi 502 et pas 500 ?****

■ Le code 500 signifie que *notre* serveur a planté. Le code 502 signifie que notre serveur a fait appel à un service tiers qui lui a répondu u

8. Recherche et filtrage — Exercice 3

On ajoute des **query parameters** à la route **GET /products** pour filtrer les résultats. Les query parameters se passent dans l'URL après le **?** :

JAVASCRIPT

```
GET /products?name=witcher
GET /products?price=30
GET /products?name=game&about=fps&price=50
```

Ils sont accessibles via **req.query**.

Construction dynamique de la requête SQL

Le défi est de construire une clause **WHERE** uniquement si des filtres sont présents. La librairie **postgres** permet de composer des fragments SQL :

```
app.get("/products", async (req, res) => {
  const { name, about, price } = req.query;

  let products;

  if (!name && !about && !price) {
    // Pas de filtre : requête simple
    products = await sql`SELECT * FROM products`;
  } else {
    // On construit la liste des conditions dynamiquement
    const conditions = [];

    if (name) conditions.push(sql`name ILIKE ${"%" + name + "%"}`);
    // ILIKE = LIKE insensible à la casse
    // %mot% = "contient ce mot"

    if (about) conditions.push(sql`about ILIKE ${"%" + about + "%"}`);

    if (price) conditions.push(sql`price <= ${parseFloat(price)}`);
    // <= : prix inférieur ou égal à la valeur

    // On fusionne toutes les conditions avec AND
    const where = conditions.reduce((a, b) => sql`${a} AND ${b}`);

    products = await sql`SELECT * FROM products WHERE ${where}`;
  }

  res.send(products);
});
```

Exemple de résultat pour **GET /products?name=cyber&price;=50** :

```
SELECT * FROM products WHERE name ILIKE '%cyber%' AND price <= 50
```

9. La ressource Orders — Exercice 4

Modèle de données

Une commande contient :

- **user_id** : l'identifiant de l'utilisateur
- **product_ids** : un tableau d'identifiants de produits
- **total** : calculé automatiquement (somme des prix × 1.2 pour la TVA)
- **payment** : booléen, **false** par défaut
- **created_at** / **updated_at** : horodatages automatiques

Calcul automatique du total

Le total est calculé côté serveur pour éviter toute manipulation côté client. On applique une TVA de 20% :

```
const getOrderTotal = async (productIds) => {
  const products = await sql`SELECT price FROM products WHERE id = ANY(${productIds})`;
  // ANY() est l'opérateur PostgreSQL pour "dans ce tableau"

  const subtotal = products.reduce((sum, p) => sum + parseFloat(p.price), 0);
  return Math.round(subtotal * 1.2 * 100) / 100; // arrondi à 2 décimales
};
```

JAVASCRIPT

POST /orders — créer une commande

```
const CreateOrderSchema = z.object({
  user_id: z.number().int().positive(),
  product_ids: z.array(z.number().int().positive()).min(1),
  payment: z.boolean().optional().default(false),
});

app.post("/orders", async (req, res) => {
  const result = CreateOrderSchema.safeParse(req.body);

  if (!result.success) {
    return res.status(400).send(result);
  }

  const { user_id, product_ids, payment } = result.data;

  // Vérification que l'utilisateur existe
  const user = await sql`SELECT id FROM users WHERE id = ${user_id}`;
  if (!user.length) {
    return res.status(404).send({ message: "User not found" });
  }

  // Vérification que tous les produits existent
  const products = await sql`SELECT id FROM products WHERE id = ANY(${product_ids})`;
  if (products.length !== product_ids.length) {
    return res.status(404).send({ message: "One or more products not found" });
  }

  const total = await getOrderTotal(product_ids);

  const order = await sql`
    INSERT INTO orders (user_id, product_ids, total, payment)
    VALUES (${user_id}, ${product_ids}, ${total}, ${payment})
    RETURNING *
  `;

  res.send(order[0]);
});
```

JAVASCRIPT

GET /orders/:id — retourner les données complètes

Par conception, le GET d'une commande doit retourner l'utilisateur et les produits **complets** (pas seulement leurs IDs) :

JAVASCRIPT

```
app.get("/orders/:id", async (req, res) => {
  const order = await sql`SELECT * FROM orders WHERE id = ${req.params.id}`;

  if (!order.length) {
    return res.status(404).send({ message: "Not found" });
  }

  // Requête séparée pour récupérer l'utilisateur (sans son mot de passe)
  const [user] = await sql`
    SELECT id, username, email FROM users WHERE id = ${order[0].user_id}
  `;

  // Requête séparée pour récupérer les produits du tableau product_ids
  const products = await sql`
    SELECT * FROM products WHERE id = ANY(${order[0].product_ids})
  `;

  // On reconstruit la réponse en remplaçant les IDs par les objets complets
  const { user_id, product_ids, ...orderData } = order[0];

  res.send({ ...orderData, user, products });
});
```

10. La ressource Reviews — Exercice 5

Modèle de données

Un avis contient :

- `user_id` et `product_id` : les références
- `score` : entier entre 1 et 5
- `content` : le texte de l'avis
- `created_at` / `updated_at` : horodatages

Mise à jour du produit à chaque modification

À chaque création, modification ou suppression d'un avis, on doit :

1. Recalculer la liste des `review_ids` du produit
2. Recalculer le `average_score` du produit

On centralise cette logique dans une fonction helper :

JAVASCRIPT

```
const updateProductReviews = async (productId) => {
  // Récupère tous les avis du produit
  const reviews = await sql`SELECT score FROM reviews WHERE product_id = ${productId}`;
  const reviewIds = await sql`SELECT id FROM reviews WHERE product_id = ${productId}`;

  // Calcule la moyenne (0 si aucun avis)
  const avgScore = reviews.length
    ? reviews.reduce((sum, r) => sum + r.score, 0) / reviews.length
    : 0;

  // Met à jour le produit
  await sql`
    UPDATE products
    SET review_ids = ${reviewIds.map((r) => r.id)},
        average_score = ${avgScore}
    WHERE id = ${productId}
  `;
};
```

POST /reviews — créer un avis

JAVASCRIPT

```
const CreateReviewSchema = z.object({
  user_id: z.number().int().positive(),
  product_id: z.number().int().positive(),
  score: z.number().int().min(1).max(5),
  content: z.string().min(1),
});

app.post("/reviews", async (req, res) => {
  const result = CreateReviewSchema.safeParse(req.body);

  if (!result.success) {
    return res.status(400).send(result);
  }

  const { user_id, product_id, score, content } = result.data;

  // Vérifications d'existence
  const user = await sql`SELECT id FROM users WHERE id = ${user_id}`;
  if (!user.length) {
    return res.status(404).send({ message: "User not found" });
  }

  const product = await sql`SELECT id FROM products WHERE id = ${product_id}`;
  if (!product.length) {
    return res.status(404).send({ message: "Product not found" });
  }

  const review = await sql`
    INSERT INTO reviews (user_id, product_id, score, content)
    VALUES (${user_id}, ${product_id}, ${score}, ${content})
    RETURNING *
  `;

  // Mise à jour du produit après création de l'avis
  await updateProductReviews(product_id);

  res.send(review[0]);
});
```

GET /products/:id avec les reviews

Depuis l'exercice 5, le GET d'un produit inclut ses avis. On fait une requête supplémentaire :

```
app.get("/products/:id", async (req, res) => {  
  const product = await sql`SELECT * FROM products WHERE id = ${req.params.id}`;  
  
  if (!product.length) {  
    return res.status(404).send({ message: "Not found" });  
  }  
  
  // On récupère tous les avis associés à ce produit  
  const reviews = await sql`SELECT * FROM reviews WHERE product_id = ${req.params.id}`;  
  
  // On enrichit la réponse avec les avis  
  res.send({ ...product[0], reviews });  
});
```

JAVASCRIPT

11. Documentation Swagger — Exercice 6

Swagger (OpenAPI) permet de générer automatiquement une page de documentation interactive à partir de commentaires dans le code.

Configuration

```
const swaggerJsdoc = require("swagger-jsdoc");  
const swaggerUi = require("swagger-ui-express");  
  
const swaggerSpec = swaggerJsdoc({  
  definition: {  
    openapi: "3.0.0",  
    info: {  
      title: "Marketplace API",  
      version: "1.0.0",  
      description: "REST API for a video game marketplace",  
    },  
  },  
  apis: [".server.js"], // fichiers à analyser pour trouver les commentaires  
});  
  
// Monte l'interface web sur la route /api-docs  
app.use("/api-docs", swaggerUi.serve, swaggerUi.setup(swaggerSpec));
```

JAVASCRIPT

Documenter une route

Les annotations Swagger s'écrivent en commentaires JSDoc au-dessus de chaque route :

JAVASCRIPT

```
/**
 * @swagger
 * /products:
 *   post:
 *     summary: Create a product
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             properties:
 *               name: { type: string }
 *               about: { type: string }
 *               price: { type: number }
 *     responses:
 *       200:
 *         description: Created product
 *       400:
 *         description: Validation error
 */
app.post("/products", async (req, res) => {
  // ...
});
```

La documentation est accessible sur : <http://localhost:8000/api-docs>

12. Résumé des bonnes pratiques

Toujours valider les entrées

Ne jamais faire confiance aux données reçues du client. Zod vérifie les types, les formats et les contraintes avant toute opération en base de données.

Toujours gérer les cas d'erreur

Chaque route doit gérer au minimum :

- **400** si les données envoyées sont invalides
- **404** si la ressource demandée n'existe pas
- **409** pour les conflits (doublons)

Ne jamais exposer des données sensibles

Le mot de passe ne doit **jamais** apparaître dans une réponse. On utilise `SELECT id, username, email` plutôt que `SELECT *` pour les utilisateurs.

Retourner la ressource modifiée

Par convention REST, après un **POST**, **PUT**, **PATCH** ou **DELETE**, on retourne toujours la ressource dans son état final. C'est ce que fait le `RETURNING *` en SQL.

Utiliser les bons codes HTTP

Situation	Code
Succès	200
Données invalides (Zod failed)	400
Ressource introuvable	404
Doublon (contrainte UNIQUE)	409
Erreur service tiers	502

Calculer les données dérivées côté serveur

Le **total** d'une commande est calculé par le serveur et non envoyé par le client. Cela évite toute manipulation des prix côté client.

Projet réalisé avec Node.js 18+, Express 5, PostgreSQL 16, Zod 4, librairie postgres 3.